

1 List Comprehension

Definition:

List comprehension provides a **short and powerful way** to create new lists from existing iterables using a single line of code.

Syntax:

```
new_list = [expression for item in iterable if condition]
```

Examples:

1 Square of numbers

```
nums = [1, 2, 3, 4, 5]
squares = [n**2 for n in nums]
print(squares) # [1, 4, 9, 16, 25]
```

2 Even numbers only

```
even = [n for n in nums if n % 2 == 0]
print(even) # [2, 4]
```

3 First letters of words

```
words = ["React", "Node", "Python"]
first_letters = [w[0] for w in words]
print(first_letters) # ['R', 'N', 'P']
```

4 Convert to uppercase

```
names = ["vamsi", "teja", "ravi"]
upper_names = [name.upper() for name in names]
print(upper_names)
```

5 Nested loop example

```
pairs = [(x, y) for x in [1, 2] for y in [3, 4]]
print(pairs) # [(1, 3), (1, 4), (2, 3), (2, 4)]
```

2 Dictionary Comprehension

Definition:

Dictionary comprehension helps to **create dictionaries in one line** by looping through an iterable.

Syntax:

```
new_dict = {key: value for item in iterable if condition}
```

Examples:

1 Number and its square

```
squares = {n: n**2 for n in range(1, 6)}  
print(squares) # {1: 1, 2: 4, 3: 9, 4: 16, 5: 25}
```

2 Convert keys to uppercase

```
data = {"name": "vamsi", "city": "vizag"}  
upper_keys = {k.upper(): v for k, v in data.items()}  
print(upper_keys)
```

3 Filter values

```
marks = {"ram": 85, "ravi": 45, "sita": 92}  
passed = {k: v for k, v in marks.items() if v >= 50}  
print(passed)
```

4 Key-value swap

```
data = {"a": 1, "b": 2, "c": 3}  
swap = {v: k for k, v in data.items()}  
print(swap)
```

5 Create dict from two lists

```
keys = ["name", "age"]  
values = ["vamsi", 22]  
result = {k: v for k, v in zip(keys, values)}  
print(result)
```

3 Set Comprehension

Definition:

Set comprehension helps to **create sets** in a single line — automatically removes duplicates.

Syntax:

```
new_set = {expression for item in iterable if condition}
```

Examples:

1 Squares of numbers

```
nums = [1, 2, 2, 3]
```

```
squares = {n**2 for n in nums}
```

```
print(squares) # {1, 4, 9}
```

2 Unique first letters

```
words = ["python", "pandas", "numpy"]
```

```
letters = {w[0] for w in words}
```

```
print(letters) # {'n', 'p'}
```

3 Filter even numbers

```
nums = [1, 2, 3, 4, 5, 6]
```

```
even = {n for n in nums if n % 2 == 0}
```

```
print(even)
```

4 Uppercase unique words

```
names = ["vamsi", "teja", "vamsi"]
```

```
unique_upper = {n.upper() for n in names}
```

```
print(unique_upper)
```

5 Characters from string

```
chars = {ch for ch in "programming"}
```

```
print(chars)
```

4 Tuple Comprehension (Generator Expression)

Definition:

Python does **not have tuple comprehension directly**, but you can use a **generator expression** that behaves similarly.

Syntax:

```
tuple_var = tuple(expression for item in iterable if condition)
```

Examples:

1 Square numbers

```
nums = (n**2 for n in range(1, 6))
```

```
print(tuple(nums))
```

2 Even numbers only

```
nums = (n for n in range(10) if n % 2 == 0)
```

```
print(tuple(nums))
```

3 Names in uppercase

```
names = ("vamsi", "teja", "ravi")  
upper_names = (n.upper() for n in names)  
print(tuple(upper_names))
```

4 Length of each word

```
words = ("React", "Node", "Python")  
lengths = (len(w) for w in words)  
print(tuple(lengths))
```

5 Square roots (approx)

```
import math  
roots = (round(math.sqrt(n), 2) for n in [4, 9, 16])  
print(tuple(roots))
```

✓ Summary:

Comprehension	Output Type	Example Use
List	[]	Filter, transform data
Dict	{key:value}	Create dictionaries
Set	{}	Get unique filtered values
Tuple (Generator)	tuple()	Memory-efficient iteration